**Software Requirements Specification (SRS)**

**Project: Hostel Management System**

**Version: 1.0**

**Team Number : 3**

**Authors:**

**<Meghana P/ PES2UG24CS642>**

**<B S Sharath Chandra Reddy/ PES2UG24CS630>**

**<ADARSHA V / PES2UG24CS621>**

**<MOHAMMADNOOR SIRASGI/PES2UG25CS812>**

**Date: 12-09-2026**

**Status: Draft**

**Revision History**

| Version | Date | Author | Change summary | Approval |
|---|---|---|---|---|
| 1.0 | 12-09-2026 | Project Team-3 | Initial SRS draft for Hostel Management System | |

**Approvals**

| Role | Name | Signature / Email | Date |
|---|---|---|---|
| Course Coordinator | | | |
| Project Guide | | | |

**Table of Contents**

**1. Introduction**

**2. Overall description**

**3. External interface requirements**

**4. System features (detailed)**

**5. Non-functional requirements (detailed)**

**6. Quality attributes & Acceptance tests**

**7. System models and diagrams**

**8. Requirements Traceability Matrix (RTM)**

# 1. Introduction

## 1.1 Purpose

**This document is a Software Requirements Specification (SRS) for a Hostel Management System (HMS). It defines the functional and non-functional requirements, external interfaces, and verification criteria for a web-based system that manages hostel room allocation, student records, fees, and related administrative operations. It is intended for use by developers, testers, and evaluators as the authoritative reference for the system to be built.**

## 1.2 Scope

**The Hostel Management System automates the day-to-day operations of a student hostel, including student registration and admission, room and bed allocation, fee collection and receipts, complaint and maintenance tracking, visitor logging, mess/meal management, notices, and administrative reporting. The system will be developed as a web application using Python (Django or Flask) for the backend, with PHP (Laravel) considered as an alternative technology stack, and a relational database for persistence. The scope excludes physical access-control hardware (biometric locks, CCTV integration) beyond optional future extension points, and excludes payroll/HR management for hostel staff.**

## 1.3 Audience

**Hostel Warden/Administrator, Hostel Students, Accounts/Finance staff, Maintenance staff, Developers, QA Engineers, and Project Evaluators.**

## 1.4 Definitions, Acronyms and Abbreviations

**HMS: Hostel Management System. SRS: Software Requirements Specification. FR: Functional Requirement. NFR: Non-Functional Requirement. RBAC: Role-Based Access Control. UI: User Interface. API: Application Programming Interface. TLS: Transport Layer Security. RTM: Requirements Traceability Matrix. ORM: Object-Relational Mapping.**

## 2. Overall description

### 2.1 Product perspective

**The HMS is a new, self-contained web application intended to replace manual/register-based hostel administration. It interacts with three principal actors — Students, Wardens/Administrators, and Accounts staff — through a browser-based UI, and persists data in a relational database (e.g., PostgreSQL/MySQL). The backend will be implemented in Python using Django (with Django REST Framework for APIs) or Flask; PHP with the Laravel framework is an acceptable alternative stack for the same architecture.**

### 2.2 Major product functions (detailed)

- **Student registration, profile management, and document upload**
- **Room and bed inventory management and allocation/de-allocation**
- **Room change/transfer requests and approvals**
- **Hostel fee structure definition, invoicing, online/offline payment recording, and receipt generation**
- **Complaint and maintenance request logging, assignment, and status tracking**
- **Visitor entry/exit logging and gate pass management**
- **Mess/meal plan management and attendance**
- **Notice board / announcements to all or selected students**
- **Leave/outing request and approval workflow**
- **Role-based user and access management (Admin, Warden, Accountant, Student)**
- **Reporting and dashboards (occupancy, dues, complaints, revenue)**

### 2.3 User roles and characteristics (expanded)

- **Student: basic computer literacy; uses the system to view room/fee details, raise complaints, and request leave or room change.**
- **Warden/Administrator: manages allocations, approves requests, oversees day-to-day hostel operations; needs quick access to occupancy and complaint dashboards.**
- **Accountant/Finance staff: manages fee structures, records payments, and generates financial reports.**
- **Maintenance Staff: receives and updates status of assigned maintenance/complaint tickets.**
- **System Administrator: manages user accounts, roles, and system configuration.**

### 2.4 Operating environment

**Server-side: Linux server running Python 3.x with Django/Flask (or PHP 8.x with Laravel), a relational database (PostgreSQL/MySQL), and a WSGI/ASGI application server (e.g., Gunicorn) behind a reverse proxy (Nginx). Client-side: any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile, with responsive UI support.**

### 2.5 Constraints

**The system must be built using Python (Django/Flask) or PHP (Laravel) as specified by the project brief. Student personal and financial data must be protected in line with institutional data-privacy policy. The system must support at least 500 concurrent student users for a mid-size hostel deployment and must be deployable on commodity/campus server infrastructure.**

## 3. External interface requirements

### 3.1 User interfaces

**A responsive web UI with role-based dashboards: Student Portal, Warden/Admin Console, and Accounts Console. The UI shall follow accessible design practices (readable contrast, keyboard navigation, descriptive labels) and shall provide clear forms for registration, allocation, fee payment, and complaint submission.**

### 3.2 Hardware interfaces

- **Standard client devices (desktop/laptop/mobile) with a web browser — no specialised hardware required for the core system.**
- **Optional future integration point for biometric/RFID gate devices for visitor and attendance logging.**

### 3.3 Software interfaces

- **Relational Database (PostgreSQL/MySQL) accessed via the framework's ORM (Django ORM / SQLAlchemy / Eloquent).**
- **Payment Gateway API (e.g., Razorpay/Stripe/PayU) for online fee payment, integrated over authenticated HTTPS/JSON.**
- **Email/SMS Gateway API for notices, payment receipts, and alerts.**
- **Institutional Student Information System (optional integration) for verifying enrolment status via a REST API.**

### 3.4 Communications interfaces

- **HTTPS (TLS 1.2+) for all client-server communication.**
- **RESTful JSON APIs between the web front end and backend services, and for third-party payment/SMS integrations.**

***[Note: this SRS defines 18 functional requirements, 5 non-functional requirements, 3 security objectives, and 5 security requirements — meeting the minimum of 15 FRs, 5 NFRs, 2 security objectives, and 5 security requirements.]***

## 4. System features (detailed)

**Each requirement below includes its acceptance criteria and a reference test case. IDs follow HMS-F-###.**

### 4.1 Student Registration & Profile Management

**Description: Allow students to register, submit personal/guardian details and documents, and allow admins to verify and approve registrations.**

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| HMS-F-001 | The system shall allow a new student to register online with personal, academic, and guardian details. | Functional | High | Student | AC-HMS-F-001: Registration form submission creates a pending student record. Test: TC-Reg-01 | Requires student DB schema |
| HMS-F-002 | The system shall allow students to upload identity and admission documents (ID proof, admission letter) in PDF/JPEG format. | Functional | Medium | Student | AC-HMS-F-002: Uploaded file stored and linked to student profile. Test: TC-Reg-02 | File size/type validation |
| HMS-F-003 | The system shall allow the Warden/Admin to review, approve, or reject a student registration with a reason. | Functional | High | Warden/ Admin | AC-HMS-F-003: Approved students transition to "Active" status; rejected students are notified. Test: TC-Reg-03 | Notification module dependency |

### 4.2 Room & Bed Allocation

**Description: Manage room/bed inventory and allocate/de-allocate beds to active students.**

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/ Dependencies |
|---|---|---|---|---|---|---|
| HMS-F-010 | The system shall maintain an inventory of rooms and beds with type, capacity, and current occupancy status. | Functional | High | Warden/Admin | AC-HMS-F-010: Room list reflects real-time occupancy counts. Test: TC-Room-01 | Core data model |
| HMS-F-011 | The system shall allow the Warden/Admin to allocate an available | Functional | High | Warden/Admin | AC-HMS-F-011: Allocation reduces available bed count by | Depends on HMS-F-003 |

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/ Dependencies |
|---|---|---|---|---|---|---|
| | bed to an approved student. | | | | one and updates student record. Test: TC-Room-02 | |
| HMS-F-012 | The system shall prevent allocation of an already-occupied bed and shall display only available beds during allocation. | Functional | High | Warden/Admin | AC-HMS-F-012: Attempting to allocate an occupied bed returns an error. Test: TC-Room-03 | Concurrency control required |
| HMS-F-013 | The system shall allow a student to submit a room change/transfer request with a reason. | Functional | Medium | Student | AC-HMS-F-013: Request appears in Warden's pending queue. Test: TC-Room-04 | |
| HMS-F-014 | The system shall allow the Warden/Admin to approve or reject a room transfer request and automatically re-allocate beds on approval. | Functional | Medium | Warden/Admin | AC-HMS-F-014: Approved transfer updates both old and new bed occupancy. Test: TC-Room-05 | Depends on HMS-F-013 |
| HMS-F-015 | The system shall de-allocate a bed automatically when a student checks out or is marked inactive. | Functional | High | Warden/Admin | AC-HMS-F-015: Bed status changes to "Available" on checkout. Test: TC-Room-06 | |

### 4.3 Fee Management

**Description: Define fee structures, generate invoices, and record/track payments.**

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| HMS-F-020 | The system shall allow the Accountant to define hostel fee structures (room rent, mess fee, security deposit) per room type and term. | Functional | High | Accountant | AC-HMS-F-020: Fee structure saved and applied to new invoices. Test: TC-Fee-01 | |
| HMS- | The system shall | Functional | High | System/Ac | AC-HMS-F-021: Invoice | Depends on |

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| F-021 | automatically generate a fee invoice for a student upon room allocation, based on the applicable fee structure. | | | countant | generated with correct line items and due date. Test: TC-Fee-02 | HMS-F-011, HMS-F-020 |
| HMS-F-022 | The system shall allow students to pay fees online via an integrated payment gateway and shall record the transaction status. | Functional | High | Student | AC-HMS-F-022: Successful payment marks invoice as "Paid" and generates a receipt. Test: TC-Fee-03 | Payment Gateway API |
| HMS-F-023 | The system shall allow the Accountant to record offline/manual fee payments (cash/cheque) against an invoice. | Functional | Medium | Accountant | AC-HMS-F-023: Manual payment updates invoice balance and issues receipt. Test: TC-Fee-04 | |
| HMS-F-024 | The system shall send an automated reminder notification for fee invoices that are unpaid within 3 days of the due date. | Functional | Medium | System | AC-HMS-F-024: Reminder email/SMS sent and logged. Test: TC-Fee-05 | Email/SMS Gateway |

### 4.4 Complaint & Maintenance Management

**Description: Allow students to log complaints/maintenance issues and track resolution.**

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| HMS-F-030 | The system shall allow a student to submit a complaint or maintenance request with category, description, and optional photo. | Functional | High | Student | AC-HMS-F-030: Complaint created with status "Open" and unique ticket ID. Test: TC-Comp-01 | |
| HMS-F-031 | The system shall allow the Warden/Admin to assign a complaint to maintenance staff and | Functional | High | Warden/Admin | AC-HMS-F-031: Status changes are timestamped and visible to the student. | Depends on HMS-F-030 |

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| | update its status (Open/In-Progress/Resolved). | | | | Test: TC-Comp-02 | |
| HMS-F-032 | The system shall notify the student when their complaint status changes. | Functional | Medium | System | AC-HMS-F-032: Notification sent on every status transition. Test: TC-Comp-03 | Notification module |

### 4.5 Visitor, Leave & Attendance Management

**Description: Track visitor entries, student leave/outing requests, and mess attendance.**

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| HMS-F-040 | The system shall allow gate/reception staff to log visitor entry and exit details against a student record. | Functional | Medium | Reception/ Warden | AC-HMS-F-040: Visitor log entry recorded with timestamp and host student. Test: TC-Vis-01 | |
| HMS-F-041 | The system shall allow a student to submit a leave/outing request specifying dates and reason, subject to Warden approval. | Functional | Medium | Student | AC-HMS-F-041: Approved leave appears on student's record and outpass is generated. Test: TC-Leave-01 | |
| HMS-F-042 | The system shall allow mess staff to mark daily meal attendance for students opting into the mess plan. | Functional | Low | Mess Staff | AC-HMS-F-042: Attendance recorded per student per meal per day. Test: TC-Mess-01 | |

### 4.6 Notices, Reporting & Administration

**Description: Broadcast notices, generate operational reports, and manage users/roles.**

| Req ID | Requirement (shall...) | Type | Priority | Source/Stakeholder | Acceptance criteria / Test case ref | Comments/Dependencies |
|---|---|---|---|---|---|---|
| HMS-F-050 | The system shall allow the Warden/Admin to publish notices/announcements visible to all students or a selected subset. | Functional | Medium | Warden/Admin | AC-HMS-F-050: Published notice appears on student dashboards immediately. Test: TC-Notice-01 | |
| HMS-F-051 | The system shall generate an occupancy report showing allocated vs. available beds per room/floor/block. | Functional | Medium | Warden/Admin | AC-HMS-F-051: Report reflects current allocation data and is exportable to PDF/CSV. Test: TC-Rep-01 | Depends on HMS-F-010 |
| HMS-F-052 | The system shall generate a fee-dues report listing students with outstanding balances. | Functional | Medium | Accountant | AC-HMS-F-052: Report lists correct outstanding amounts and is exportable. Test: TC-Rep-02 | Depends on HMS-F-021 |
| HMS-F-053 | The system shall allow the System Administrator to create, edit, deactivate, and assign roles (Admin, Warden, Accountant, Student, Maintenance) to user accounts. | Functional | High | System Admin | AC-HMS-F-053: Role changes take effect immediately and restrict/allow access accordingly. Test: TC-Admin-01 | RBAC module |

## 5. Non-functional requirements (detailed)

**NFRs below are measurable and tied to test plans. IDs HMS-NF-###.**

| Req ID | Requirement | Category | Priority | Acceptance criteria / Measurement |
|---|---|---|---|---|
| HMS-NF-001 | 95th percentile page response time for common student and warden operations (search, view, submit) shall be ≤ 3 seconds under normal load. | Performance | High | Load test shows p95 ≤ 3s with 200 concurrent users. Test: TC-Perf-01 |
| HMS-NF-002 | The system shall provide 99.5% availability monthly, excluding scheduled maintenance windows. | Reliability | High | Uptime monitoring report shows ≥ 99.5% per month. Test: Ops reports |
| HMS-NF-003 | The system shall be usable by students and staff with minimal training, achieving a System Usability Scale (SUS) score ≥ 70. | Usability | Medium | SUS survey conducted with pilot users. Test: TC-UX-01 |
| HMS-NF-004 | The system architecture shall support scaling to at least 2,000 registered students and 500 concurrent sessions without redesign. | Scalability | Medium | Load/capacity test at target scale passes. Test: TC-Scale-01 |
| HMS-NF-005 | The system shall retain audit logs of allocation, fee, and access-control events for at least 3 years for institutional audit purposes. | Audit/Data Retention | Medium | Log retention policy verified in staging/production. Test: TC-Ops-01 |

## 5.1 Security

### 5.1.1 Security Objectives

- **Protect the confidentiality and integrity of student personal, academic, and financial data against unauthorised access or disclosure.**
- **Ensure that only authenticated and appropriately authorised users can perform allocation, fee, and administrative actions (least privilege / RBAC).**
- **Ensure accountability by maintaining tamper-evident audit trails of security-relevant actions (logins, allocations, payments, role changes).**

### 5.1.2 Security Requirements

| Req ID | Requirement (shall...) | Type | Priority | Acceptance criteria / Test case ref |
|---|---|---|---|---|
| HMS-SR-001 | TLS 1.2+ shall be mandatory for all network connections between clients, the application server, and third-party APIs. | Security | High | AC-HMS-SR-001: All endpoints reject plain-HTTP connections. Test: TC-Sec-01 |
| HMS-SR-002 | The system shall store user passwords using a salted, one-way hashing algorithm (e.g., bcrypt/Argon2); plaintext passwords shall never be stored or logged. | Security | High | AC-HMS-SR-002: Database inspection shows only hashed passwords. Test: TC-Sec-02 |
| HMS-SR-003 | The system shall enforce role-based access control (RBAC) so that Students cannot access Warden/Admin or Accountant functions. | Security | High | AC-HMS-SR-003: Access attempts to unauthorised endpoints return HTTP 403. Test: TC-Sec-03 |
| HMS-SR-004 | The system shall automatically terminate an inactive user session after 15 minutes and require re-authentication. | Security | Medium | AC-HMS-SR-004: Session expires and protected pages redirect to login after timeout. Test: TC-Sec-04 |
| HMS-SR-005 | The system shall log all authentication attempts, role/permission changes, and fee/payment transactions to a tamper-evident audit log. | Security | High | AC-HMS-SR-005: Audit log entries are immutable (append-only) and reviewable by admins. Test: TC-Sec-05 |

## 6. Quality attributes & Acceptance tests

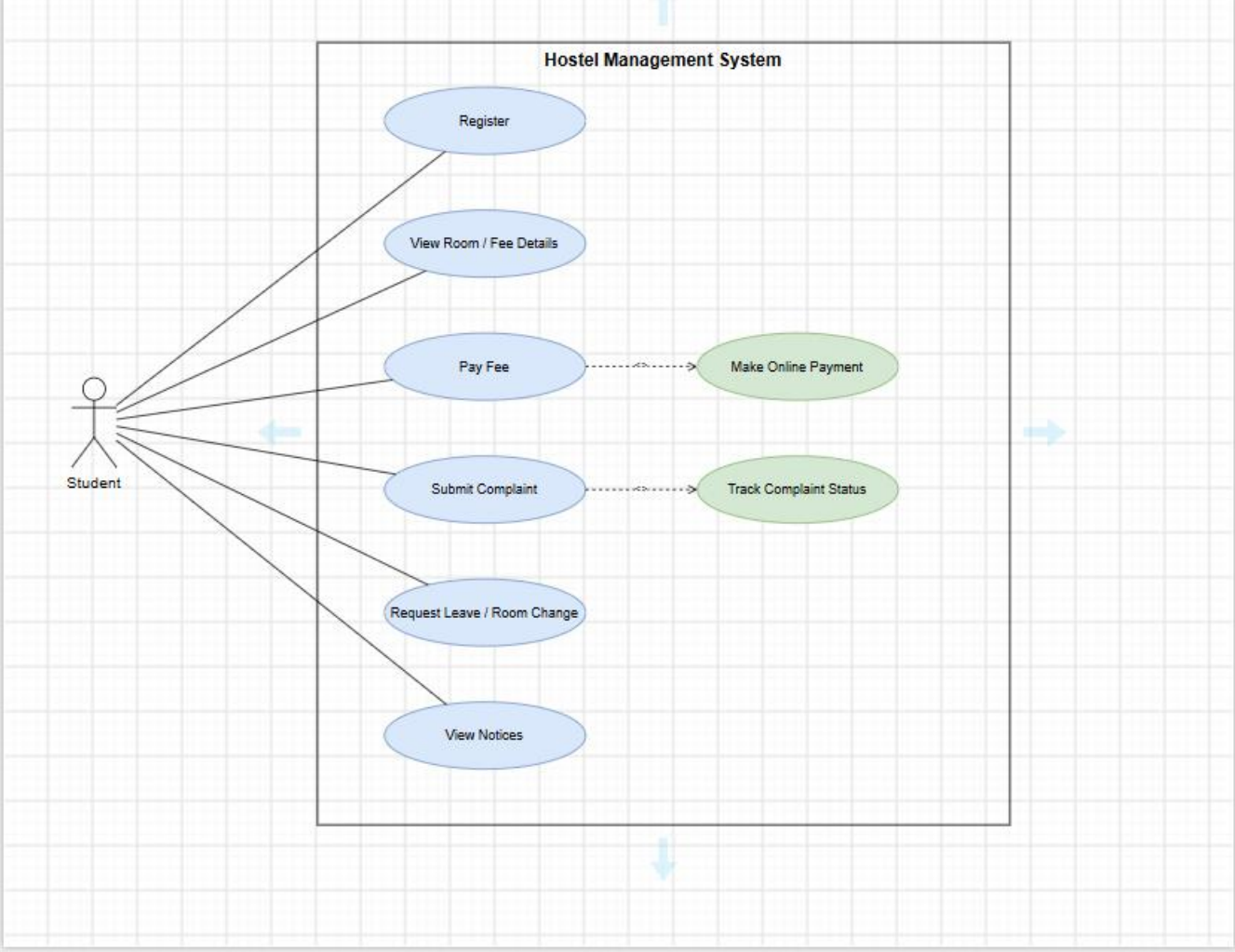
**Exit criteria for acceptance: all High-priority functional requirements are implemented and verified, no critical NFR failures remain open, all security requirements pass their acceptance tests, and the RTM shows all associated test cases passed.**

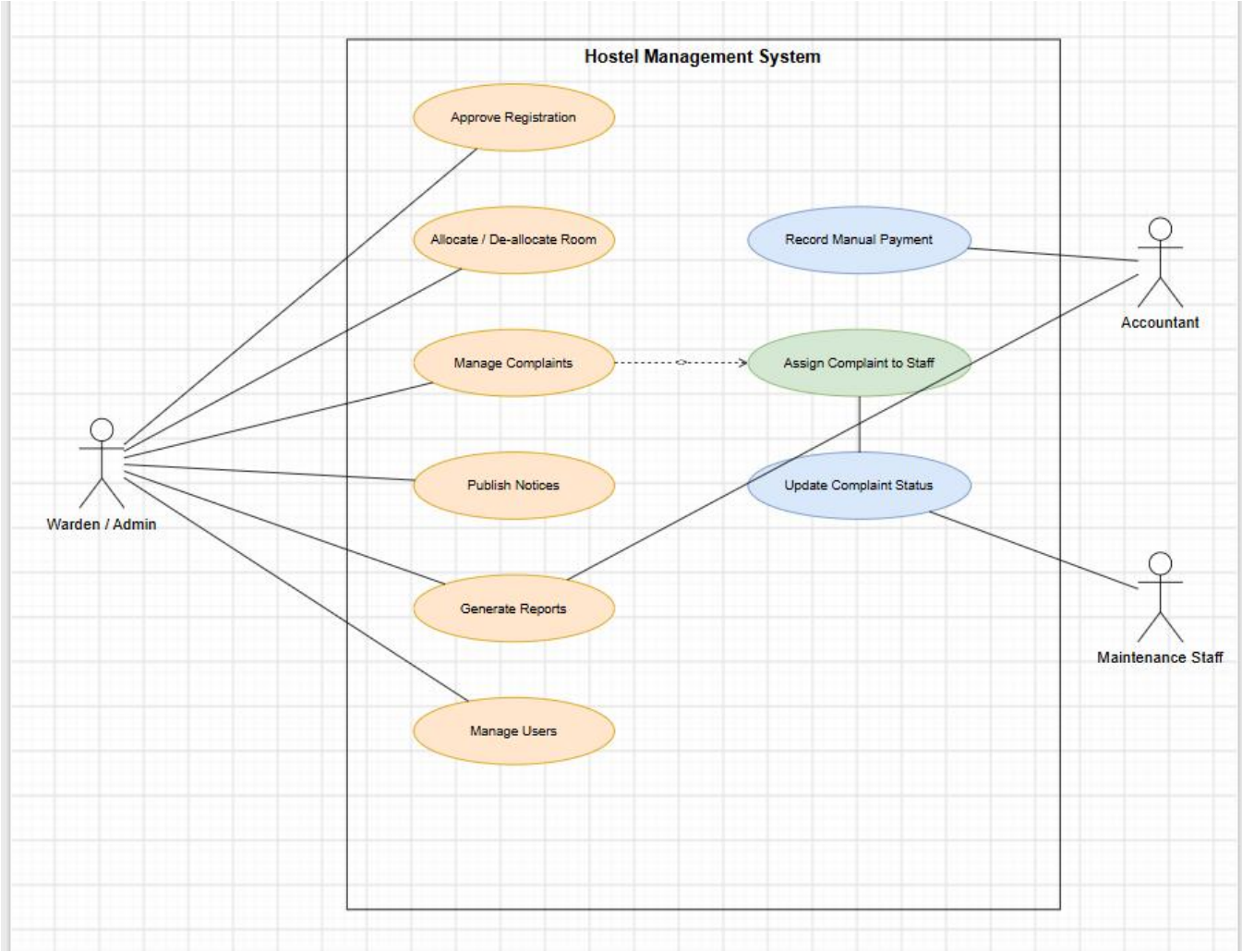
**Acceptance test suites: Registration, Room Allocation, Fee Management, Complaint Management, Visitor/Leave Management, Notices & Reporting, Performance, Security, and Usability.**

## 7. System models and diagrams

### 7.1 UML Use-Case diagrams

**At least two UML use-case diagrams shall be produced during design: (1) Student use cases — Register, View Room/Fee Details, Pay Fee, Submit Complaint, Request Leave/Room Change; (2) Warden/Admin use cases — Approve Registration, Allocate/De-allocate Room, Manage Complaints, Publish Notices, Generate Reports, Manage Users. Diagrams are to be attached as an appendix or maintained in the project design document.**




Hostel Management System
Approve Registration
Allocate / De-allocate Room
Manage Complaints
Publish Notices
Generate Reports
Manage Users
Record Manual Payment
Assign Complaint to Staff
Update Complaint Status
Warden / Admin
Accountant
Maintenance Staff

## 8. Requirements Traceability Matrix (RTM)

| Req ID | Requirement (short) | Section ref / Design Spec | Module | Test case(s) | Status (N/P/A) | Comments |
|---|---|---|---|---|---|---|
| HMS-F-001 | Student registration | 4.1 / DS-Reg-01 | RegistrationModule | TC-Reg-01 | N | |
| HMS-F-011 | Allocate bed | 4.2 / DS-Room-01 | AllocationModule | TC-Room-02 | N | |
| HMS-F-021 | Auto-generate invoice | 4.3 / DS-Fee-01 | FeeModule | TC-Fee-02 | N | |
| HMS-F-030 | Submit complaint | 4.4 / DS-Comp-01 | ComplaintModule | TC-Comp-01 | N | |
| HMS-F-053 | Manage user roles | 4.6 / DS-Admin-01 | UserAdminModule | TC-Admin-01 | N | |
| HMS-NF-001 | Response time target | 5 / DS-Perf-01 | WebUI / Backend API | TC-Perf-01 | N | |
| HMS-SR-002 | Password hashing | 5.1.2 / DS-Sec-01 | AuthModule | TC-Sec-02 | N | |
| HMS-SR-003 | RBAC enforcement | 5.1.2 / DS-Sec-02 | AuthModule | TC-Sec-03 | N | |